

INTERNSHIP PROJECT REPORT

on

DAILY DASHBOARD

Submitted By:

Name: Govind

Roll No.: 2200680310042

Course: B.Tech (E.CE)

Semester: VII

College: Meerut Institute of Engineering & Technology

Internship Details:

Organization: Sopra Steria Ltd.

Internship Duration: September 2025 – November 2025

Project Title: *Daily Dashboard (Weather Forecast, Stocks & BBC News Portal)*

Submitted To:

Developed by: Govind

****Under the guidance of Sopra Steria Ltd.***

Date of Submission: 2025

Daily Dashboard

- Complete Project Guide & Presentation

Executive Summary

The Daily Dashboard is a Python-based web application designed to provide real-time data visualization and reporting capabilities. Built with a modern web framework architecture, it features user authentication, database management, and automated report generation.

Project Overview

Purpose

A comprehensive dashboard application that allows users to monitor, track, and generate reports on daily operations and metrics.

Key Capabilities

- User authentication and access control
 - Real-time data visualization
 - Automated report generation
 - Database management and queries
 - Responsive web interface
-

Detailed Project Structure:

```
DAILY_DASHBOARD/
├── __pycache__/           # Python bytecode cache (auto-generated)
├── .venv/                 # Virtual environment (isolated
dependencies)
├── static/               # Frontend assets
├── Templates/            # HTML view templates
├── .env                  # Environment configuration (secrets)
├── .env.example          # Environment template
├── app.py                # Main application
├── dashboard.db          # SQLite database
├── database_setup.sql    # Database schema
├── generate_report.py     # Report generator
├── requirements.txt       # Dependencies list
├── README.md             # Documentation
├── Test Files/
│   ├── test_db.py        # Database tests
│   ├── test_login.py     # Login tests
│   └── test_postgres.py  # PostgreSQL tests
```

Core Components Deep Dive

1. app.py - Application Entry Point

Purpose: Main Flask application server

Functionality:

- Initializes the web server
- Defines URL routes and endpoints
- Handles HTTP requests and responses
- Manages middleware and session handling
- Coordinates between templates, database, and business logic

Key Features:

- Route management (/login, /dashboard, /reports, etc.)
- Request validation and error handling
- Integration with authentication system
- Database connection management

2. dashboard.db - SQLite Database

Purpose: Primary data storage

Contains:

- User credentials and profiles
- Dashboard metrics and data points
- Historical records for reporting
- Application settings and configurations

Characteristics:

- Lightweight and file-based
- No separate server required
- Perfect for development and small-to-medium deployments
- Easy backup (single file)

3. database_setup.sql - Database Schema

Purpose: Defines database structure

Contents:

- Table creation statements (CREATE TABLE)
- Column definitions and data types
- Primary and foreign key relationships

- Indexes for query optimization
- Initial data seeding (default users, settings)

Example Structure:

```
sql

CREATE TABLE users (
  id INTEGER PRIMARY KEY,
  username TEXT UNIQUE,
  password_hash TEXT,
  created_at TIMESTAMP
);

CREATE TABLE dashboard_metrics (
  id INTEGER PRIMARY KEY,
  metric_name TEXT,
  value REAL,
  timestamp TIMESTAMP
);
```

4. Templates/ - HTML Views

Purpose: User interface templates

Contains:

- login.html - Authentication page
- dashboard.html - Main dashboard view
- reports.html - Report display page
- base.html - Base template (header, footer, navigation)

Technology: Server-side rendering with Jinja2 (Flask) or Django templates

Features:

- Dynamic content injection
- Template inheritance
- Form rendering and validation
- Responsive design

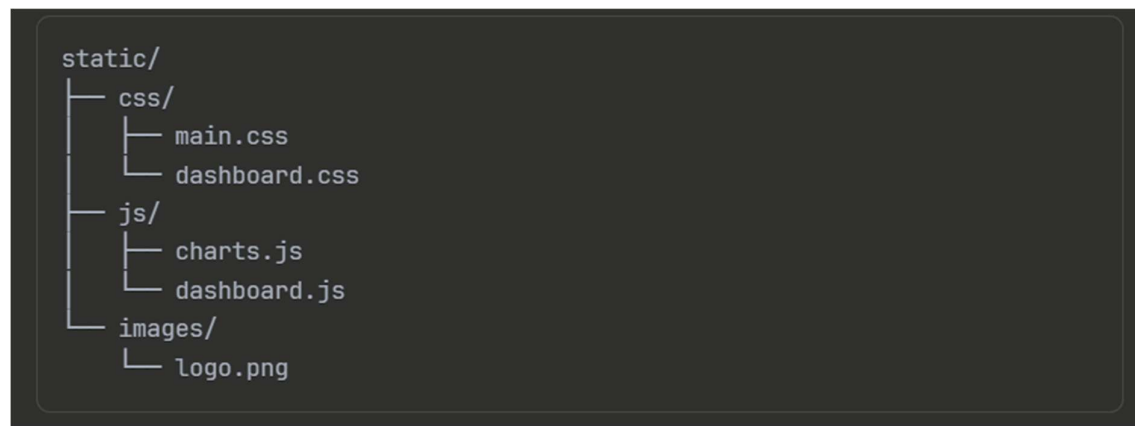
5. static/ - Static Assets

Purpose: Client-side resources

Contains:

- **CSS:** Stylesheets for design and layout
- **JavaScript:** Interactive functionality
- **Images:** Logos, icons, graphics
- **Fonts:** Custom typography

Organization:



6. generate_report.py - Report Generator

Purpose: Automated report creation

Functionality:

- Queries database for specific metrics
- Aggregates and processes data
- Generates formatted reports (PDF, Excel, CSV)
- Can be scheduled or triggered manually
- Email integration for automated distribution

Use Cases:

- Daily/weekly/monthly summaries
- Performance metrics reports
- Data exports for analysis
- Custom report generation based on filters

7. .env - Environment Configuration

Purpose: Secure configuration management

Contains:

- Database credentials
- Secret keys for encryption
- API keys for external services
- Debug mode settings
- Email server configuration

Example:

```
DATABASE_URL=sqlite:///dashboard.db
SECRET_KEY=your-secret-key-here
DEBUG=True
MAIL_SERVER=smtp.gmail.com
```

8. requirements.txt - Dependencies

Purpose: Python package management

Contains:

```
Flask==2.3.0
SQLAlchemy==2.0.0
pandas==2.0.0
matplotlib==3.7.0
python-dotenv==1.0.0
pytest==7.3.0
```

Usage: pip install -r requirements.txt

Testing Suite

test_db.py - Database Testing

Tests:

- Database connection establishment
- CRUD operations (Create, Read, Update, Delete)
- Data integrity and constraints
- Query performance
- Transaction rollback scenarios

test_login.py - Authentication Testing

Tests:

- Valid login credentials
- Invalid login attempts
- Password hashing and verification
- Session management
- Access control and permissions
- Logout functionality

test_postgres.py - PostgreSQL Integration

Tests:

- PostgreSQL connection
- Migration from SQLite to PostgreSQL
- Database compatibility
- Query syntax differences
- Performance under load

How the Application Works :

Application Flow

1. User Access

- User navigates to the application URL
- app.py serves the login page from Templates/
- Static assets loaded from static/

2. Authentication

- User submits credentials
- app.py validates against dashboard.db
- Session created upon successful login
- User redirected to dashboard

3. **Dashboard View**

- app.py queries dashboard.db for metrics
- Data processed and sent to dashboard.html template
- JavaScript in static/js/ renders charts and graphs
- Real-time updates via AJAX requests

4. **Report Generation**

- User requests a report
- generate_report.py called by app.py
- Data extracted from dashboard.db
- Report formatted and returned to user
- Option to download or email

5. **Data Management**

- New data added through forms or API
- app.py validates and inserts into dashboard.db
- Dashboard automatically updates

Technology Stack

Backend :

- **Python 3.x:** Core programming language
- **Flask/Django:** Web framework
- **SQLAlchemy:** Database ORM (Object-Relational Mapping)
- **SQLite:** Development database
- **PostgreSQL:** Production database option

Frontend :

- **HTML5:** Structure
- **CSS3:** Styling
- **JavaScript:** Interactivity
- **Chart.js/Plotly:** Data visualization
- **Bootstrap:** Responsive design

Development Tools :

- **Virtual Environment:** Dependency isolation
 - **pytest:** Testing framework
 - **python-dotenv:** Environment management
-

Setup & Installation

Prerequisites :

- Python 3.8+
- pip (Python package manager)

Steps :

1. Clone Repository

```
cd DAILY_DASHBOARD
```

2. Create Virtual Environment

```
python -m venv .venv
```

```
source .venv/bin/activate # On Windows: .venv\Scripts\activate
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. Configure Environment

```
cp .env.example .env
```

```
# Edit .env with your configuration
```

5. Initialize Database

```
sqlite3 dashboard.db < database_setup.sql
```

6. Run Application

```
python app.py
```

7. Access Dashboard

Navigate to <http://localhost:5000>

Usage Scenarios

Scenario 1: Daily Operations Monitoring

- Login to dashboard
- View real-time metrics and KPIs
- Identify trends and anomalies
- Take action based on insights

Scenario 2: Report Generation

- Navigate to Reports section
- Select date range and metrics
- Generate report
- Download as PDF or schedule email delivery

Scenario 3: Data Entry

- Access data entry forms
 - Input new metrics or records
 - Validate and submit
 - Data immediately reflected in dashboard
-

Security Features :

1. **Password Hashing:** Secure credential storage
 2. **Session Management:** Prevents unauthorized access
 3. **Environment Variables:** Sensitive data protection
 4. **Input Validation:** SQL injection prevention
 5. **HTTPS Support:** Encrypted data transmission
-

Future Enhancements :

- Real-time notifications and alerts
 - Mobile application
 - Advanced analytics and ML predictions
 - Multi-user role management
 - API for third-party integrations
 - Cloud deployment (AWS/Azure/GCP)
-

Maintenance & Support :

Regular Tasks

- Database backups
- Log monitoring
- Performance optimization
- Security updates
- Dependency updates

Monitoring

- Application uptime
 - Response times
 - Error rates
 - User activity
-

Conclusion :

The Daily Dashboard is a robust, scalable web application designed for efficient data management and visualization. Its modular architecture ensures easy maintenance and future enhancements, while comprehensive testing guarantees reliability and security.

Key Strengths

- ✓ Clean, maintainable code structure
 - ✓ Comprehensive testing suite
 - ✓ Secure authentication system
 - ✓ Flexible reporting capabilities
 - ✓ Scalable architecture
 - ✓ Well-documented codebase
-